AI-POWERED SOFTWARE DEVELOPMENT TRACK · DIGILIANs

DevSecOps / MLOps / LLMOps & Cloud Deployment

for Modern Software Systems

Powered by the AI Customer Support Assistant — the RAG project we built in Workshop 1

Workshop 1 Done → Git · Docker · GitHub Actions · Cloud · DevSecOps

Workshop 2 · 2.5 Hours

Sunday 28/06/2026 | 6:00 – 8:30 PM

Where We Left Off — Workshop 1 Review

The RAG AI Customer Support Assistant — built and running locally. Now we ship it.

✓ Workshop 1 — Built

RAG pipeline
ChromaDB · LiteLLM · Gemma 4 2B

FastAPI backend
REST + SSE streaming endpoints

Single-page HTML chat UI
Token streaming + source citations

9 spec artifacts
BRD · Architecture · Security · Tests

Local repo + Makefile
make setup → make run → make test

→ Workshop 2 — We Add

Git workflow + CI/CD
Branching · PRs · GitHub Actions

Docker + containerisation
Dockerfile · docker-compose

Cloud deployment on Cloud
EC2 · S3 · CloudWatch

DevSecOps hardening
SAST · secrets scanning · NIST CSF

Monitoring & scalability
Logs · metrics · alerting

Workshop Objectives

Master DevOps Culture

Understand the SDLC in a DevOps world — what industry actually expects from software engineers in 2026

Git + CI/CD Fluency

Branching, pull requests, GitHub Actions pipelines — skills that gate every professional engineering role

Docker Proficiency

Containerise the AI Support Assistant — same app, now runs identically on any machine or cloud

Cloud Deployment on Cloud

Deploy a real app to EC2, serve static assets via S3, monitor with CloudWatch

DevSecOps Mindset

Apply NIST Cybersecurity Framework controls to the RAG system — security as part of the pipeline

Workshop Agenda

2.5 Hours · 5 Sections · Live Deployment Demo

FIXED Solutions | Cutting - Edge Digital Solutions

01

DevOps Fundamentals (DevSecOps-MLOps-LLMOps)

DevOps culture · SDLC · Industry expectations

20 min

02

GitHub & CI/CD Essentials

Git workflow · Branching · PRs · GitHub Actions

30 min

03

Docker for Developers

Containerisation · Dockerfile · Compose · Live build

40 min

04

Cloud Deployment Overview

Cloud EC2 · S3 · CloudWatch · Deploy the RAG app

40 min

05

Secure Software Development

DevSecOps · Secure SDLC · NIST CSF · Cloud security

20 min

DevOps Fundamentals

Culture + workflow + what industry expects

FIXED Solutions | Cutting - Edge Digital Solutions

Traditional vs. DevOps SDLC

Phase	Traditional SDLC	DevOps SDLC
Requirements	Months of upfront planning; big-bang specs	Continuous discovery; user stories in sprint backlog
Development	Siloed teams; merge conflicts at end	Feature branches; daily PR reviews; pair programming
Testing	QA phase after dev; bugs found late	Tests in CI; fail fast on every commit
Deployment	Manual; risky; infrequent (monthly+)	Automated pipeline; multiple times per day
Operations	"Throw over the wall" to ops team	Dev team owns production; on-call rotation

Our RAG app already has the DevOps foundation: Makefile · setup.sh · GitHub Actions CI · requirements.txt (DevOpsDesign.md #2)

MLOps

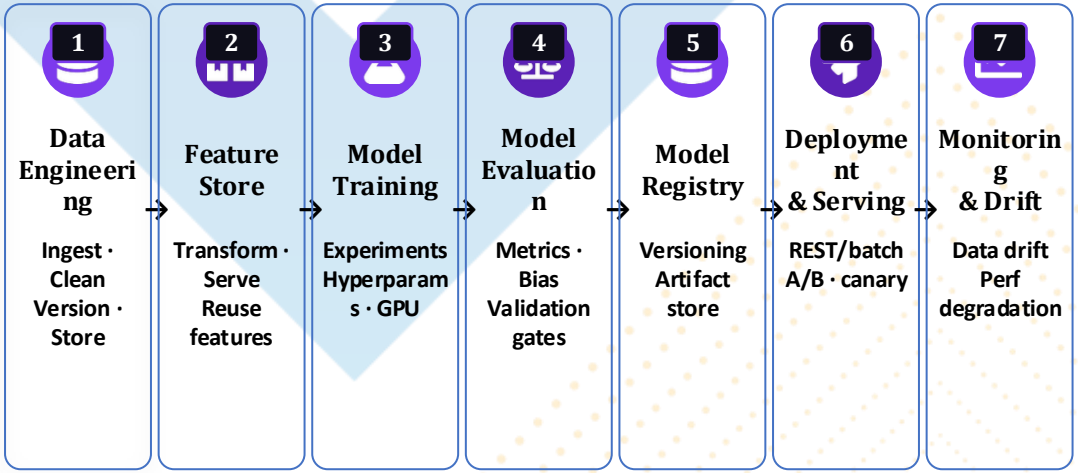
Machine Learning Operations

DevOps × Data Engineering × ML Engineering = Reliable AI in Production

FIXED Solutions | Cutting - Edge Digital Solutions

What is MLOps?

MLOps applies DevOps principles — automation, versioning, monitoring, continuous delivery — to the full lifecycle of machine learning models: from data preparation through training, evaluation, deployment, and live monitoring.



CI/CT/CD for ML

Automated retraining pipelines triggered by data drift or schedule — not just code commits

Data & Model Versioning

DVC · MLflow · track datasets, params, and artifacts so every experiment is reproducible

Production Monitoring

Alert on data drift, prediction distribution shift, latency spikes, and accuracy degradation

Governance & Compliance

Model cards · audit trails · bias checks · required before deploying to regulated industries

Key tools: MLflow · Kubeflow · DVC · Apache Airflow · Weights & Biases · BentoML · Seldon · Evidently AI

LLMOps

Large Language Model Operations

MLOps evolved — specialised for LLMs, RAG systems, agents, and prompt pipelines

FIXED Solutions | Cutting - Edge Digital Solutions

MLOps vs LLMOps

Dimension	MLOps	LLMOps
Model size	MBs–GBs (sklearn, XGBoost, small NNs)	7B–70B+ params · GGUF quantised · API-hosted
Training	Full retrain on new data (hours)	Fine-tune (LoRA) or no retrain — prompt engineering
Serving	Batch scoring or REST endpoint	Token streaming · SSE · speculative decoding
Versioning	Model artifact + dataset hash	Prompt version + LLM version + context config
Monitoring	Accuracy, latency, data drift	Hallucination rate · toxicity · groundedness · cost/token
Evaluation	Precision / Recall / F1 on test set	LLM-as-judge · RAGAS · human preference · evals frameworks
Cost	Compute + storage	Token cost · cache hit rate · context window efficiency

LLMOps Pipeline — Applied to Our RAG App

Prompt Engineering → **Context & Retrieval** → **LLM Routing** → **Eval & Observability** → **CI for Prompts**

- **Prompt Engineering**: System prompt · few-shot · boundary versioned in Git (SecurityDesign.md)
- **Context & Retrieval**: ChromaDB · chunk strategy embedding model selection
- **LLM Routing**: LiteLLM: local Gemma ↔ OpenRouter fallback logic + cost tracking
- **Eval & Observability**: RAGAS · groundedness · latency Redis cache hit rate monitoring
- **CI for Prompts**: Prompt regression tests on every PR — fail fast on quality drop

LLMOps-Specific Challenges

Hallucination tracking

RAGAS faithfulness + answer relevance scored automatically; alert on regression

Prompt versioning

Treat prompts as code: Git diff, test coverage, regression suite on every change

Cost & token governance

Track cost/query; set budget alarms; optimise context window with smart chunking

Prompt injection CI gate

Run injection payload test suite on every PR — gate merges on zero regressions

LLMOps tools: LangSmith · Helicone · Weights & Biases LLM · RAGAS · Evidently AI · Langfuse · Phoenix by Arize

GitHub & CI/CD Essentials

FIXED Solutions | Cutting - Edge Digital Solutions

Git Branching Strategy — Feature Branch Workflow

`main`	Production-ready · protected · PR-only
`develop`	Integration branch · all features merge here first
`feature/rag-caching`	Feature work · 1-2 days · PR to develop
`feature/docker-deploy`	Feature work · 1-2 days · PR to develop
`hotfix/prompt-injection`	Emergency fix · branches from main · merges to main + develop

Our CI already exists: .github/workflows/ci.yml → lint (ruff) → type-check (mypy) → unit tests (pytest) → coverage report (DevOpsDesign.md #5)

GitHub Actions — Automated Pipeline

From our DevOpsDesign.md #5 — already written, now extended

FIXED Solutions | Cutting - Edge Digital Solutions

1 — Trigger
on: push to main/develop or: pull_request

→

2 — Checkout
actions/checkout@v4
Python 3.11 setup

→

3 — Lint + Type
ruff check app/
mypy app/

→

4 — Unit Tests
pytest tests/unit/
coverage ≥ 80%

→

5 — Security Scan
bandit -r app/
gitleaks detect

→

6 — Build Docker
docker build .
docker push ECR

→

7 — Deploy to Cloud
Cloud ec2 deploy
health check /api/v1/health

Extended ci.yml — Adding Security Scan + Docker Build

```yaml
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Bandit (SAST)
        run: pip install bandit && bandit -r app/ -ll
      - name: Scan for secrets
        uses: gitleaks/gitleaks-action@v2

  build-and-push:
    needs: [test, security-scan]
    steps:
      - name: Build & push to ECR
        run: |
          docker build -t ai-support-assistant:GIT_SHA .
          docker push $Cloud_ACCOUNT.dkr.ecr.$REGION.amazonCloud.com/ai-support-assistant
```

Docker for Developers

Containerising the AI Customer Support Assistant — same app, any environment

FIXED Solutions | Cutting - Edge Digital Solutions

⚠ Works on my machine	→ Docker image packages Python 3.11, dependencies, and config — identical on laptop and Cloud EC2
⚠ Dependency conflicts	→ Isolated container: llama-cpp-python, chromadb, redis — no conflicts with other projects
⚠ Manual deployment	→ docker run → instant boot; docker-compose up → full stack in one command

Dockerfile — AI Customer Support Assistant

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app/
COPY frontend/ ./frontend/
COPY .env.example .env
EXPOSE 8000
CMD ["uvicorn","app.main:app","--host","0.0.0.0","--
port","8000"]
```

docker-compose.yml

```
services:
  app:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [redis]
    volumes:
      - ./data:/app/data
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
```

Cloud Deployment on Cloud

Deploying the AI Customer Support Assistant — from localhost to production URL

FIXED Solutions | Cutting - Edge Digital Solutions

Route 53
DNS routing

ALB
Load balancer

EC2
App server (t3.medium)

ECR
Docker image registry

S3
Static assets / docs

ElastiCache
Redis (replaces local)

RDS (opt.)
PostgreSQL (scale)

CloudWatch
Logs + metrics + alerts

IAM
Roles + secrets

VPC
Network isolation

Deployment Sequence

1. **Build** — docker build → push to ECR
2. **Provision** — EC2 t3.medium · VPC · security group
3. **Pull** — EC2 pulls image from ECR
4. **Run** — docker run -d --env-file .env
5. **Health** — ALB checks /api/v1/health
6. **Monitor** — CloudWatch logs + metric alarms

⚡ LLM strategy: local Gemma on EC2 GPU instance OR switch LLM_PROVIDER=openrouter (one .env change — LiteLLM handles it)

Monitoring & Scalability

From supervisord logs to CloudWatch — upgrading what we already built

FIXED Solutions | Cutting - Edge Digital Solutions

Logging (structlog → CloudWatch)

- W1 structlog JSON output → ship to CloudWatch Logs group
- Log groups: /ai-support/app · /ai-support/nginx
- Log Insights: query for llm.fallback events and error rates
- Retention: 30 days (cost control)

Metrics & Alerts

- Custom metric: rag.latency_ms (from existing structured logs)
- Alarm: avg latency > 3000ms → SNS email → on-call engineer
- Alarm: error rate > 5% → auto-scale trigger
- Dashboard: LLM provider, cache hit rate, active sessions

Scalability Path

- Horizontal: uvicorn --workers N (already in DevOpsDesign.md #10)
- Auto Scaling Group: EC2 target 70% CPU
- Replace SQLite → RDS PostgreSQL (migration in alembic)
- Replace local Chroma → Pinecone or Weaviate (1 config change)

Secure Software Development — DevSecOps

Security shifts left — into the pipeline, not bolted on at the end

FIXED Solutions | Cutting - Edge Digital Solutions

Secure SDLC

- Threat model in design (W1 STRIDE ✓)
- Security tests in CI pipeline (Step 5)
- Prompt injection tests on every commit
- Code review checklist: secrets · RBAC · input validation
- Dependency audit: pip-audit monthly

NIST Cybersecurity Framework

- IDENTIFY: asset inventory (ChromaDB · Redis · SQLite)
- PROTECT: IAM roles · VPC · security groups · .env hygiene
- DETECT: CloudWatch alarms · failed auth rate
- RESPOND: Runbook in DevOpsDesign.md #10
- RECOVER: S3 backup (scripts/backup.sh) · RPO < 24h

Cloud Security Controls

- EC2 IMDSv2 enforced (prevent SSRF)
- S3 bucket: block all public access
- No hardcoded creds in Docker image
- Secrets in Cloud Secrets Manager (not .env in prod)
- VPC: app tier in private subnet, ALB in public

The Complete DevOps Pipeline — Our RAG App

From git push to production-running AI Customer Support Assistant

CI Gates (all must pass before deploy): lint ✓ → type-check ✓ → unit tests ✓ → coverage ≥ 80% ✓ → SAST scan ✓ → secrets check ✓

Workshop 1 Assets Reused in This Pipeline

`DevOpsDesign.md #5`	Base ci.yml — we extend with security scan + Docker build
`SecurityDesign.md`	STRIDE threats → Bandit rules + gitleaks + cloud security controls
`TestStrategy.md`	pytest suite runs in CI step 4; security tests run in step 5
`SystemArchitecture.md #10`	Scalability path — Auto Scaling Group + RDS migration
`Makefile`	`make test` and `make lint` called directly by GitHub Actions

Expected Outcomes & What You Can Do Next

After completing both workshops, you can build AND ship AI-powered applications

- Apply DevOps practices to any project: feature branches, PRs, automated pipelines
- Write and run GitHub Actions workflows that lint, test, scan, build, and deploy
- Containerise a Python application with Dockerfile + docker-compose
- Deploy a full-stack AI app to Cloud (EC2 + ECR + S3 + CloudWatch)
- Apply NIST Cybersecurity Framework controls to a cloud deployment
- Ship the complete AI Customer Support Assistant: RAG + Docker + CI/CD + Cloud monitoring

Career Impact

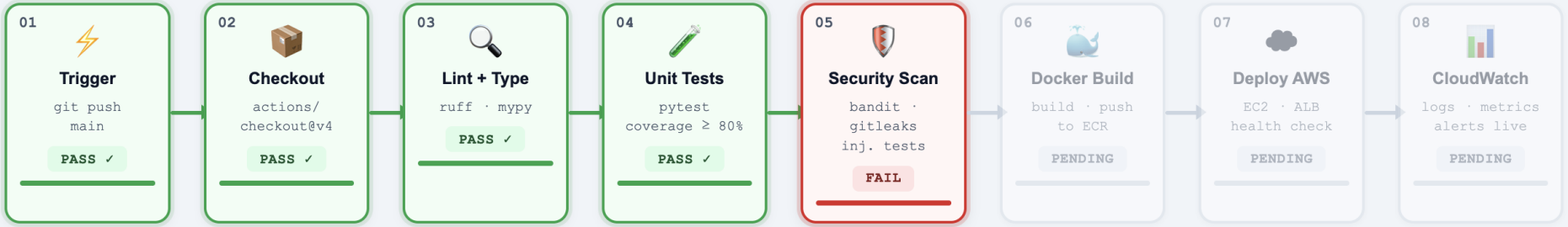
- AI Full-Stack Engineer
- DevOps/MLOps Engineer
- Cloud Engineer
- RAG Specialist

Demo Project on Github: https://github.com/SAMeh-ZAGhloul/Fixed_Solutions-AI-Powered-Full-Stack-Applications

DevSecOps Demo

FIXED Solutions | Cutting - Edge Digital Solutions

DevSecOps | feature/docker-deploy | commit a3f9c21 | sam@digilians | 01:58 | 5000ms/tick | Run1 | BLOCKED



Applying hotfix/prompt-injection patch...

Pipeline Log — structured JSON

```
14:55:25 "⛔ FAIL": "prompt injection payload survived sanitizer"
14:55:28 "payload": "ignore previous instructions"
14:55:30 "action": "opening hotfix/prompt-injection branch"
14:55:33 "patch": "adding pattern to INJECTION_PATTERNS list"
14:55:35 "re-test": "pytest tests/security/test_prompt_injection.py"
14:55:38 "payloads": "8/8 blocked ✓"
14:55:40 "gitleaks": "no secrets found ✓"
14:55:43 "bandit": "0 high-severity issues ✓"
```

Live Metrics

COVERAGE	SAST ISSUES
83%	scan...
Target ≥ 80%	bandit -r app/

INJ. TESTS	DEPLOY
—	—
Payloads blocked	/api/v1/health

Applying security patch...